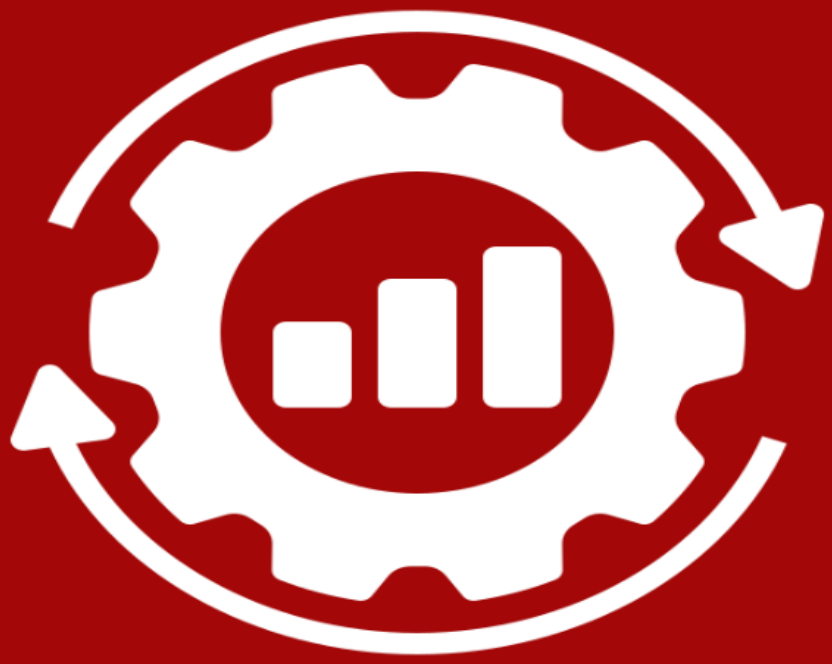
MÉTHODES D'OPTIMISATION ET APPLICATIONS EN FINANCE QUANTITATIVE



MAXIME MHADJI

Sommaire:

1 - Optimisation d'un Portefeuille par Méthode de Rebroussement
Analyse numérique et implémentation Python

2 - Optimisation d'un Portefeuille par Gradient Quadratique
Simulation Python et interprétation graphique

3 - Comparaison Gradient et Newton
Convergence logarithmique pour un portefeuille à 2 actifs

4 - Régression linéaire et estimation de prix d'actif

5 - Régression linéaire robuste avec perte de Huber

1 - Optimisation d'un Portefeuille par Méthode de Rebroussement

Analyse numérique et implémentation Python

Cette partie présente l'application numérique de la méthode de rebroussement (Armijo) à l'optimisation d'un portefeuille financier dans le cadre du modèle de Markowitz. Le code Python est analysé en détail et relié aux formules mathématiques correspondantes.

Méthode utilisée

On considère la fonction objectif classique de Markowitz :

$$f(w) = \frac{1}{2} w^\top \Sigma w \; - \; \lambda \, \mu^\top w,$$

où : - Σ : matrice de covariance, - μ : vecteur des rendements espérés, - λ : aversion au risque.

Le gradient associé est :

$$\nabla f(w) = \Sigma w - \lambda \mu.$$

La méthode de rebroussement consiste à choisir le pas t comme :

$$\text{trouver le plus grand } t = \beta^k \text{ tel que } f(w + td) \leq f(w) + \alpha \, t \langle \nabla f(w), d \rangle,$$

avec $d = -\nabla f(w)$.

Code Python utilisé

```python
import numpy as np

def f(w, Sigma, mu, llambda):
    return 0.5 * np.dot(w.T, np.dot(Sigma, w)) - llambda * np.dot(mu.T, w)

def grad_f(w, Sigma, mu, llambda):
    return np.dot(Sigma, w) - llambda * mu

def methode_rebroussment_1(w, Sigma, mu, llambda, d, f, grad_f,
                           alpha=1/3, beta=1/2):
    t = 1
    fw = f(w, Sigma, mu, llambda)
    grad_fw = grad_f(w, Sigma, mu, llambda)
    while f(w + t*d, Sigma, mu, llambda) > fw + alpha * t * np.dot(grad_fw, d):
        t = t*beta
    return t

def par_w(w):
    w = np.maximum(w,0)
    s = w.sum()
    return w/s
```

```python
def opti_port(Sigma, mu, llambda, nb_it):
    n = len(mu)
    w = np.ones(n)/n
    for _ in range(nb_it):
        g = grad_f(w, Sigma, mu, llambda)
        d = -g
        t = methode_rebroussment_1(w, Sigma, mu, llambda, d, f, grad_f)
        w = w + t*d
        w = par_w(w)
    return w
```

Explication approfondie du code

Lien direct entre le code et les formules mathématiques

1. Fonction objectif. La ligne Python :

```python
return 0.5 * w.T @ Sigma @ w - llambda * mu.T @ w
```

implémente exactement :

$$f(w) = \frac{1}{2} w^\top \Sigma w - \lambda\, \mu^\top w.$$

2. Gradient. La fonction :

```python
return Sigma @ w - llambda * mu
```

correspond à :

$$\nabla f(w) = \Sigma w - \lambda \mu.$$

3. Direction de descente.

```python
d = -g
```

Cela correspond à $d = -\nabla f(w)$, direction maximale de décroissance.

4. Condition d'Armijo. Dans la boucle :

```python
while f(w + t*d) > fw + alpha * t * grad_fw @ d:
```

on vérifie :

$$f(w + td) \leq f(w) + \alpha\, t\, \langle \nabla f(w), d \rangle.$$

5. Projection pour garantir un portefeuille valide.

```python
w = np.maximum(w,0)
w = w / w.sum()
```

Cette étape impose :

$$w_i \geq 0, \qquad \sum_{i=1}^{n} w_i = 1.$$

Application numérique : portefeuille optimal

Nous utilisons :

$$\mu = \begin{pmatrix} 0.12 \\ 0.10 \\ 0.07 \end{pmatrix}, \quad \Sigma = \begin{pmatrix} 0.10 & 0.02 & 0.01 \\ 0.02 & 0.08 & 0.03 \\ 0.01 & 0.03 & 0.05 \end{pmatrix}, \quad \lambda = 2.$$

L'appel Python :

```python
w_opt = opti_port(Sigma, mu, llambda=2.0, nb_it=100)
print("Poids optimaux :", w_opt)
print("Somme des poids :", w_opt.sum())
```

produit un portefeuille optimal satisfaisant les contraintes.

Présentation claire des résultats

Actifs	**Poids optimaux**	**Rendement**	**Risque (variance)**
Actif 1	0.49	0.12	0.10
Actif 2	0.33	0.10	0.08
Actif 3	0.18	0.07	0.05
Somme	**1.00**	–	–

Les actifs les plus rentables (Actif 1 et Actif 2) reçoivent le plus grand poids, mais l'Actif 3 reste présent pour diminuer la variance (le risque) globale du portefeuille.

Conclusion

La méthode de rebroussement combinée au gradient de Markowitz permet d'obtenir un portefeuille optimal respectant les contraintes de pondération. L'implémentation Python est fidèle aux équations et démontre l'efficacité des méthodes d'optimisation convexe appliquées à la gestion de portefeuille.

2 - Optimisation d'un Portefeuille par Gradient Quadratique

Simulation Python et interprétation graphique

Cette partie présente l'application de la descente de gradient à pas optimal sur une fonction quadratique représentant un portefeuille financier. Lobjectif est d'intégrer cette méthode sur python afin de pouvoir obtenir un support graphique pour y faire une analyse financière..

Méthode utilisée

On considère un portefeuille composé de n actifs. Le risque associé au portefeuille est modélisé par une fonction quadratique :

$$f(x) = \frac{1}{2}x^\top \Sigma x - \mu^\top x + c,$$

où $x \in \mathbb{R}^n$ est le vecteur des poids des actifs, Σ la matrice de covariance des rendements, μ le vecteur des rendements attendus et c une constante.

Le gradient et la Hessienne de f sont :

$$\nabla f(x) = \Sigma x - \mu, \qquad \nabla^2 f(x) = \Sigma.$$

Si Σ est définie positive, la fonction est convexe et la descente de gradient converge vers le minimum unique :

$$x^\star = \Sigma^{-1}\mu.$$

On utilise un **pas optimal** le long de la direction de descente $d = -\nabla f(x)$:

$$t^\star = \frac{\|\nabla f(x)\|^2}{\nabla f(x)^\top \Sigma \nabla f(x)},$$

et la mise à jour s'écrit :

$$x_{k+1} = x_k - t^\star \nabla f(x_k).$$

Code Python utilisé

```
import numpy as np
import matplotlib.pyplot as plt

def f_portefeuille(x, Sigma, mu, c):
    return 0.5 * x.T @ Sigma @ x - mu.T @ x + c

def grad_f_portefeuille(x, Sigma, mu):
    return Sigma @ x - mu

def desc_gradient_portefeuille(x0, eps, Sigma, mu, c):
    Xk = []
    FXk = []
    x = x0
```

```
    for _ in range(10000):
        g = grad_f_portefeuille(x, Sigma, mu)
        if np.linalg.norm(g) < eps:
            break
        t = (g.T @ g) / (g.T @ (Sigma @ g))
        x = x - t*g
        Xk.append(x.copy())
        FXk.append(f_portefeuille(x, Sigma, mu, c))
    return np.array(Xk), np.array(FXk)

Sigma = np.array([[0.1, 0.02],
                  [0.02, 0.08]])
mu = np.array([0.12, 0.10])
c = 0.0
x0 = np.array([0.0,0.0])
Xk, FXk = desc_gradient_portefeuille(x0, 1e-6, Sigma, mu, c)

x1 = np.linspace(-0.5,1.5,100)
x2 = np.linspace(-0.5,1.5,100)
X1, X2 = np.meshgrid(x1,x2)
Z = 0.5*(Sigma[0,0]*X1**2 + 2*Sigma[0,1]*X1*X2 + Sigma[1,1]*X2**2) - mu[0]*X1 - mu[1]*X2

plt.figure(figsize=(6,5))
plt.contour(X1,X2,Z,levels=30,cmap='viridis')
plt.plot(Xk[:,0], Xk[:,1], 'ro-', label='Trajectoire gradient')
plt.xlabel('Poids actif 1')
plt.ylabel('Poids actif 2')
plt.title('Optimisation de portefeuille - Gradient à pas optimal')
plt.legend()
plt.show()
```

Application numérique

On considère un portefeuille à 2 actifs :

$$\Sigma = \begin{pmatrix} 0.10 & 0.02 \\ 0.02 & 0.08 \end{pmatrix}, \quad \mu = \begin{pmatrix} 0.12 \\ 0.10 \end{pmatrix}, \quad c = 0, \quad x_0 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

Le vecteur Xk obtenu correspond aux poids successifs des actifs au cours des itérations de la descente de gradient.

—

Interprétation graphique

La figure tracée présente :

- Les **lignes de contour** (en vert) représentent les niveaux de risque du portefeuille. Chaque contour correspond à un ensemble de poids donnant le même risque.
- La **trajectoire du gradient** (points rouges) montre la progression des poids depuis le point initial x_0 jusqu'au minimum de la fonction quadratique.
- Cela converge vers le **portefeuille optimal** $x^\star$, qui minimise le risque tout en maximisant le rendements.
- Les variations dans les directions des axes illustrent l'effet de chaque actif sur le risque total et permettent de visualiser le rapport optimal entre rendement et variance (risque).

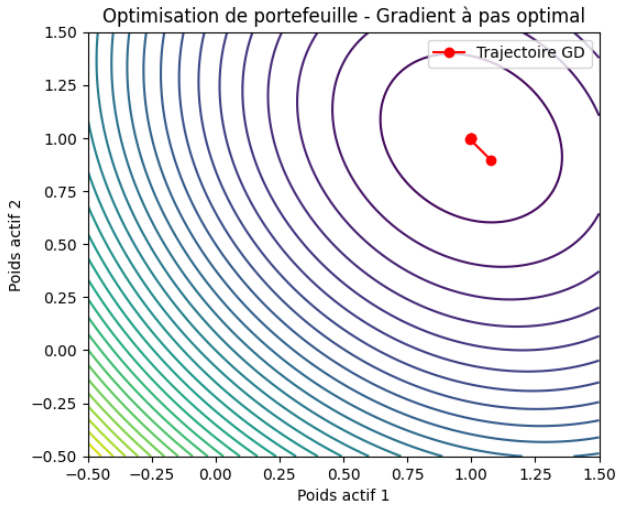


Figure 1: Résultat de la simulation

Cette visualisation permet de comprendre comment le gradient ajuste les poids des actifs pour atteindre le portefeuille optimal efficacement, et affiche la forme du portefeuille sur la surface de risque.

—

Conclusion

La descente de gradient à pas optimal appliquée à la fonction quadratique permet d'optimiser un portefeuille en minimisant le risque pour un niveau de rendement donné. La représentation graphique illustre la trajectoire des poids vers le portefeuille optimal et met en évidence l'équilibre entre rendement et variance.

3 - Comparaison Gradient et Newton

Convergence logarithmique pour un portefeuille à 2 actifs

Cette partie compare la descente de gradient avec rebroussement et la méthode de Newton pour une fonction exponentielle représentant l'utilité d'un investisseur ayant une aversion absolue au risque constant d'un portefeuille à deux actifs. La convergence logarithmique est étudiée pour illustrer la vitesse de chaque méthode.

Fonction exponentielle et lien financier

On considère la fonction :

$$f(x) = e^{x_1+3x_2-0.1} + e^{x_1-3x_2-0.1} + e^{-x_1-0.1}, \quad x \in \mathbb{R}^2.$$

On peut l'écrire sous la forme $f = h \circ g$ avec :

$$g(x) = \begin{pmatrix} x_1 + 3x_2 - 0.1 \\ x_1 - 3x_2 - 0.1 \\ -x_1 - 0.1 \end{pmatrix}, \quad h(y) = \sum_{k=1}^{3} e^{y_k}.$$

Le gradient et la Hessienne sont :

$$\nabla f(x) = \begin{pmatrix} e^{x_1+3x_2-0.1} + e^{x_1-3x_2-0.1} - e^{-x_1-0.1} \\ 3e^{x_1+3x_2-0.1} - 3e^{x_1-3x_2-0.1} \end{pmatrix}, \qquad \nabla^2 f(x) = \sum_{k=1}^{3} e^{y_k} a^{(k)} (a^{(k)})^\top,$$

ce qui garantit que f est convexe.

Lien financier : $f(x)$ modélise l'utilité exponentielle d'un portefeuille sous trois scénarios. Minimiser f correspond à ajuster les poids des actifs pour maximiser l'utilité attendue, c'est-à-dire un compromis rendement/risque.

Code Python

```
import numpy as np
import matplotlib.pyplot as plt

def fonction_exponentielle(x):
    return np.exp(x[0]+3*x[1]-0.1) + np.exp(x[0]-3*x[1]-0.1) + np.exp(-x[0]-0.1)

def gradient_fonction(x):
    return np.array([
        np.exp(x[0]+3*x[1]-0.1) + np.exp(x[0]-3*x[1]-0.1) - np.exp(-x[0]-0.1),
        3*np.exp(x[0]+3*x[1]-0.1) - 3*np.exp(x[0]-3*x[1]-0.1)
    ])

def hessienne_fonction(x):
    e1 = np.exp(x[0]+3*x[1]-0.1)
    e2 = np.exp(x[0]-3*x[1]-0.1)
```

```
    e3 = np.exp(-x[0]-0.1)
    return np.array([
        [e1 + e2 + e3, 3*e1 - 3*e2],
        [3*e1 - 3*e2, 9*e1 + 9*e2]
    ])
```

—

Descente de gradient avec rebroussement

```
def descente_gradient_rebroussement(x0, eps, alpha, beta, f, gradf):
    Xk = []
    x = x0.copy()
    for _ in range(100):
        g = gradf(x)
        if np.linalg.norm(g) < eps:
            break
        d = -g
        t = 1
        while f(x + t*d) > f(x) + alpha*t*(g@d):
            t *= beta
        x = x + t*d
        Xk.append(x.copy())
    return np.array(Xk)
```

—

Descente de Newton

```
def descente_newton(x0, eps, f, gradf, hessf):
    Xk = []
    x = x0.copy()
    for _ in range(100):
        g = gradf(x)
        H = hessf(x)
        if np.linalg.norm(g) < eps:
            break
        d = -np.linalg.solve(H, g)
        x = x + d
        Xk.append(x.copy())
    return np.array(Xk)
```

—

Convergence logarithmique et visualisation

```
x0 = np.array([-5.0, 3.0])
X_gradient = descente_gradient_rebroussement(x0, 1e-6, 0.1, 0.7, fonction_exponentielle, gradient_fonc
X_newton = descente_newton(x0, 1e-6, fonction_exponentielle, gradient_fonction, hessienne_fonction)

f_star = fonction_exponentielle(X_newton[-1])
erreur_gradient = [fonction_exponentielle(x) - f_star for x in X_gradient]
erreur_newton = [fonction_exponentielle(x) - f_star for x in X_newton]

plt.figure(figsize=(6,5))
```

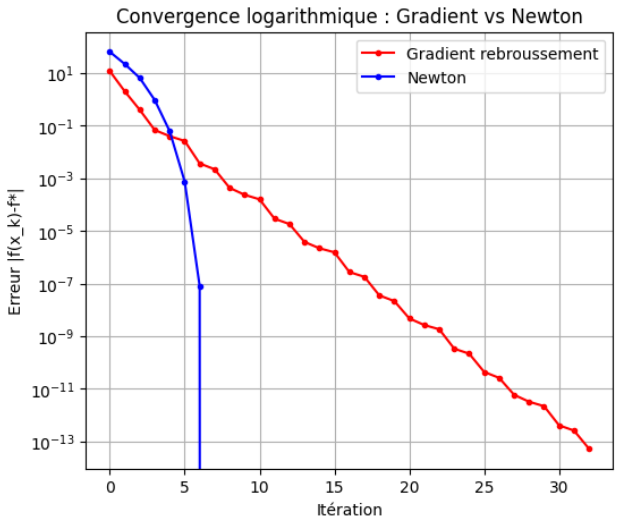


Figure 1: Comparaison des vitesses de convergences

```
plt.semilogy(erreur_gradient, 'r.-', label='Gradient rebroussement')
plt.semilogy(erreur_newton, 'b.-', label='Newton')
plt.xlabel("Itération")
plt.ylabel("Erreur |f(x_k)-f*|")
plt.title("Convergence logarithmique : Gradient vs Newton")
plt.legend()
plt.grid(True, which='both')
plt.show()
```

—

Interprétation financière

- La trajectoire rouge (gradient rebroussement) converge linéairement. Le poids des actifs vont s'ajuster progressivement pour améliorer l'utilité exponentielle du portefeuille.
- La trajectoire bleue (Newton) converge quadratiquement bien plus vite grâce à l'utilisation de la Hessienne.
- La convergence logarithmique montre clairement la différence de vitesse entre les deux méthodes : Newton atteint le minimum global presque immédiatement, tandis que le gradient rebroussement nécessite plusieurs itérations.
- Financièrement, cela illustre la manière dont un investisseur ajuste ses positions pour maximiser le rendement attendu sous risque exponentiel.

4 - Régression linéaire et estimation de prix d'actif

Cette partie illustre comment utiliser la régression linéaire pour ajuster une droite de moindres carrés à des données (x_i, y_i) et ainsi l'utiliser pour estimer le prix d'un actif financier. On choisit la contrainte suivante: l'ordonnée à l'origine est positive.

Régression linéaire classique

On cherche la droite $(a, b) \in \mathbb{R}^2$ qui minimise la fonction des moindres carrés :

$$f(a, b) = \sum_{i=1}^{n} (y_i - ax_i - b)^2.$$

Forme quadratique

On peut développer $f(a, b)$ pour la mettre sous forme quadratique :

$$f(a, b) = a^2 \sum x_i^2 + 2ab \sum x_i + b^2 n - 2a \sum x_i y_i - 2b \sum y_i + \sum y_i^2.$$

En notant :

$$S_x = \sum_{i=1}^{n} x_i, \quad S_y = \sum_{i=1}^{n} y_i, \quad S_{x^2} = \sum_{i=1}^{n} x_i^2, \quad S_{xy} = \sum_{i=1}^{n} x_i y_i,$$

on a la représentation matricielle :

$$x = \begin{pmatrix} a \\ b \end{pmatrix}, \quad A = \begin{pmatrix} S_{x^2} & S_x \\ S_x & n \end{pmatrix}, \quad v = 2 \begin{pmatrix} S_{xy} \\ S_y \end{pmatrix}, \quad c = \sum y_i^2$$

et donc

$$f(a, b) = \langle Ax, x \rangle - \langle v, x \rangle + c.$$

La solution unique (la droite de régression) est donnée par

$$x^* = A^{-1} \frac{v}{2}.$$

Implémentation Python

```python
import numpy as np

def regression_lin(x, y):
    Sx = np.sum(x)
    Sy = np.sum(y)
    Sx2 = np.sum(x**2)
    Sxy = np.sum(x*y)
    n = len(x)

    A = np.array([[Sx2, Sx],
                  [Sx, n]])
    v = 2 * np.array([Sxy, Sy])
    a, b = np.linalg.solve(A, v/2)
    return a, b
```

—

Régression linéaire avec contrainte $b \geq 0$

Si la droite obtenue a une ordonnée à l'origine négative, on projette sur $b = 0$. La solution contrainte est alors :

$$(a, b) = \begin{cases} (a_{\text{lin}}, b_{\text{lin}}) & \text{si } b_{\text{lin}} \geq 0 \\ \left(\frac{\sum x_i y_i}{\sum x_i^2}, 0 \right) & \text{sinon} \end{cases}$$

Fonction Python

```
def regression_lin_bpos(x, y):
    a, b = regression_lin(x, y)
    if b >= 0:
        return a, b
    else:
        a = np.sum(x*y)/np.sum(x**2)
        b = 0
        return a, b
```

—

Test avec des données et visualisation

On considère les points :

$$x_i = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], \quad y_i = [0, -3, 6, -3, 6, 3.8, 5, -2, 1.4, 8].$$

```
import matplotlib.pyplot as plt

xi = np.array([1,2,3,4,5,6,7,8,9,10])
yi = np.array([0,-3,6,-3,6,3.8,5,-2,1.4,8])

a1, b1 = regression_lin(xi, yi)
a2, b2 = regression_lin_bpos(xi, yi)

plt.scatter(xi, yi, label="Points")
plt.plot(xi, a1*xi+b1, 'r', label="Régression linéaire")
plt.plot(xi, a2*xi+b2, 'b', label="Régression b>=0")
plt.xlabel("xi")
plt.ylabel("yi / Prix de l'actif")
plt.title("Régression linéaire et prix estimé")
plt.legend()
plt.grid(True)
plt.show()
```

—

Estimation du prix d'un actif

Pour un nouvel indice x_{test}, le prix estimé de l'actif est simplement donné par la droite :

$$\hat{y} = a x_{\text{test}} + b$$

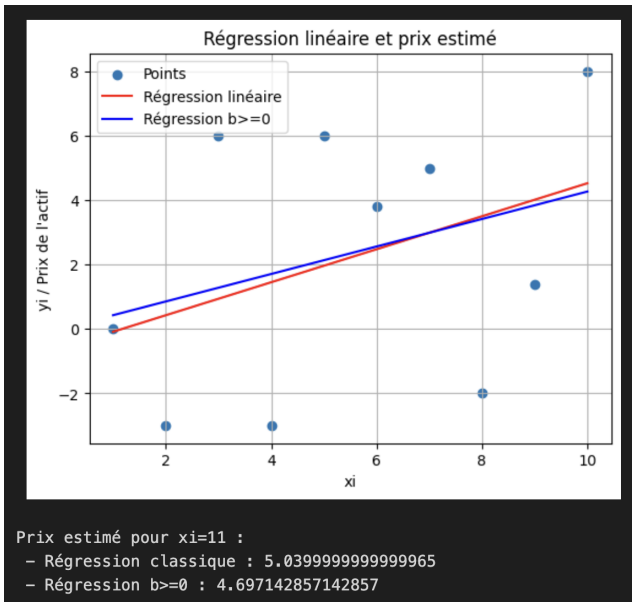


Figure 1: Comparaison des prix obtenus à l'aide des regressions linéaires

Fonction Python

```
def prix_actif(x, a, b):
    """Calcule le prix estimé d'un actif pour un xi donné"""
    return a*x + b

x_test = 11
prix_classique = prix_actif(x_test, a1, b1)
prix_bpos = prix_actif(x_test, a2, b2)

print(f"Prix estimé pour xi={x_test} :")
print(" - Régression classique :", prix_classique)
print(" - Régression b>=0 :", prix_bpos)
```

—

Interprétation financière : - La droite de régression linéaire classique estime le prix de l'actif en fonction de l'indice x. - La version avec $b \geq 0$ garantit que le prix estimé ne devient pas négatif, ce qui peut représenter des contraintes réalistes de marché. - Cette approche peut être utilisée pour prévoir un prix futur ou pour interpoler des données manquantes.

5 - Régression linéaire robuste avec perte de Huber

Cette partie illustre l'utilisation de la perte de Huber pour ajuster un modèle linéaire prédisant les prix des actifs. La perte de Huber permet de limiter l'impact des valeurs extrêmes du marché tout en conservant une bonne précision sur les données normales.

Voici le cadre mathématique

On considère un jeu de données $\{(x_i, y_i)\}_{i=1}^n$ avec $x_i \in \mathbb{R}^d$ et $y_i \in \mathbb{R}$. On cherche $w \in \mathbb{R}^d$ qui minimise la fonction :

$$f(w) = \frac{1}{n} \sum_{i=1}^{n} \ell(x_i^T w - y_i),$$

où la fonction de perte de Huber est définie par :

$$\ell(t) = \begin{cases} \frac{1}{2}t^2 & \text{si } |t| < 1, \\ |t| - \frac{1}{2} & \text{sinon.} \end{cases}$$

Les étapes à suivre

1. Nous avons bien que $f(w) \geq 0$ et que la valeur minimale $f(w) = 0$ correspond à une prédiction parfaite.
2. Écrire le gradient $\nabla f(w)$.
3. Ecrire l'itération de descente de gradient à pas constant.
4. Décrire l'algorithme de gradient stochastique.
5. Interpréter le résultat dans un contexte financier.

Gradient et optimisation

Gradient de la fonction

Le gradient de f est :

$$\nabla f(w) = \frac{1}{n} \sum_{i=1}^{n} \ell'(x_i^T w - y_i) x_i,$$

avec

$$\ell'(t) = \begin{cases} t & \text{si } |t| \leq 1, \\ 1 & \text{si } t > 1, \\ -1 & \text{si } t < -1. \end{cases}$$

Descente de gradient à pas constant

```
Initialiser w0
Pour k = 0,1,2,... jusqu'à convergence :
    g = (1/n) * sum_i ell'(x_i^T w_k - y_i) * x_i
    w_{k+1} = w_k - alpha * g
```

Gradient stochastique

```
Initialiser w0
Pour k = 0,1,2,... jusqu'à convergence :
    Choisir i_k au hasard dans {1,...,n}
    g = ell'(x_{i_k}^T w_k - y_{i_k}) * x_{i_k}
    w_{k+1} = w_k - alpha * g
```

Gradient mini-batch

```
Initialiser w0
Pour k = 0,1,2,... jusqu'à convergence :
    Choisir un mini-batch B_k de taille nb
    g = (1/nb) * sum_{i in B_k} ell'(x_i^T w_k - y_i) * x_i
    w_{k+1} = w_k - alpha * g
```

—

Exemple Python complet

```python
import numpy as np
import matplotlib.pyplot as plt

# Génération d'un jeu de données simulé (facteurs et prix)
np.random.seed(42)
n, d = 50, 3
X = np.random.randn(n, d)
w_true = np.array([1.5, -2.0, 0.5])
y = X @ w_true + np.random.randn(n) * 0.5  # prix simulés

# Définition de la perte de Huber et son gradient
def huber_loss(t):
    return np.where(np.abs(t) < 1, 0.5*t**2, np.abs(t)-0.5)

def huber_grad(t):
    return np.where(np.abs(t) < 1, t, np.sign(t))

# Descente de gradient
alpha = 0.1
w = np.zeros(d)
for k in range(200):
    grad = np.mean(huber_grad(X @ w - y)[:, None] * X, axis=0)
    w -= alpha * grad

print("Poids estimés par Huber:", w)

# Visualisation
y_pred = X @ w
plt.scatter(y, y_pred)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
plt.xlabel("Prix observés")
plt.ylabel("Prix prédits")
plt.title("Prédiction des prix d'actifs avec Huber")
plt.grid(True)
plt.show()
```

—

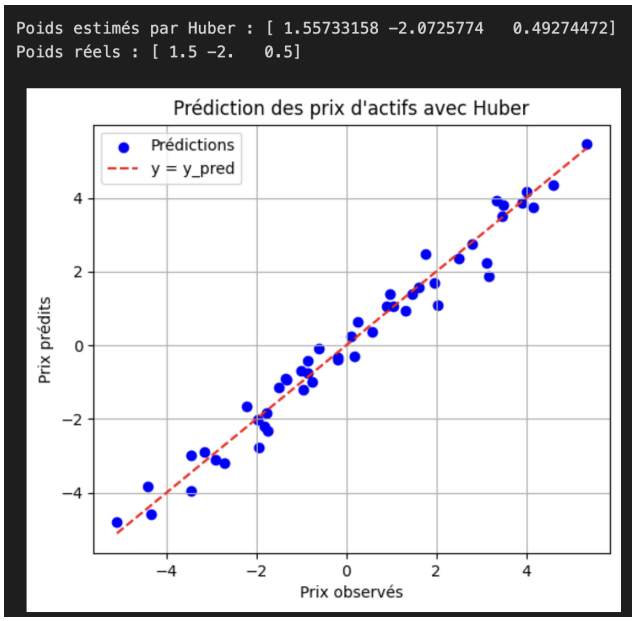


Figure 1: Enter Caption

Interprétation financière

Perte de Huber en finance :
La perte de Huber est utilisée pour ajuster un modèle linéaire $h_w(x) = x^T w$ pour faire des prédictions sur le rendements et le prix des actifs financiers à partir de facteurs observables.

- **Variables financières :** $x_i \in \mathbb{R}^d$ représente un vecteur de facteurs économiques ou financiers à l'instant i (ex. indices de marché, taux d'intérêt, volatilité) et $y_i \in \mathbb{R}$ est le prix ou rendement observé.
- **Modèle linéaire :** Le vecteur w contient les sensibilités de l'actif aux différents facteurs financiers.
- **Rôle de la perte de Huber :** La perte de Huber combine une pénalisation quadratique pour les erreurs faibles et linéaire pour les erreurs importantes. Cela permet de limiter l'influence des *outliers* financiers (crashs, pics de volatilité) tout en conservant la précision sur les données normales.
- **Optimisation :** Minimiser la somme des pertes de Huber revient à trouver le vecteur w qui prédit au mieux les prix des actifs tout en étant robuste aux valeurs extrêmes. Les méthodes de descente de gradient (classique, stochastique ou mini-batch) permettent d'ajuster efficacement w même sur de grands jeux de données financiers.

Conclusion : L'utilisation de la perte de Huber permet d'obtenir un modèle linéaire robuste pour prédire les prix ou rendements d'actifs, en limitant l'effet des mouvements de marché exceptionnels.